CINEVAULT

Comprehensive Entertainment Information, Review, Recommendation & Analytics Platform

Product Requirements Document (PRD)

Document Version	v2.0 (Final)
Project Type	Final Year Project
Platform Type	IMDb + OTT Style Web Application
Tech Stack	Flask · MySQL · Vanilla JS
ML Components	Content-Based & Collaborative Filtering
Document Purpose	College Submission & Viva Preparation

1. Executive Summary

1.1 Product Overview

CineVault is a full-stack, database-driven web application designed to function as a comprehensive entertainment information platform — combining the functionality of IMDb's movie catalogue with the discovery and watchlist features of modern OTT platforms like Netflix and Hotstar. Built using Python Flask, MySQL, and Vanilla JavaScript, CineVault delivers a seamless, data-rich experience for movie discovery, user reviews, watchlist management, and intelligent recommendations.

1.2 Problem Statement

The entertainment landscape is fragmented. Users must visit multiple platforms to find movie information, read community reviews, manage their watch queues, and discover new content. Existing platforms like IMDb are information-heavy but do not provide OTT-style watchlist tracking with status management. Meanwhile, OTT apps focus on their own catalogue. CineVault bridges this gap by consolidating these features into a single, open platform with no catalogue restrictions.

Specifically, CineVault solves the following problems:

- No single open platform combines structured movie data, community reviews, watchlist tracking, and ML-based personalization.
- Cold-start users receive generic recommendations; CineVault's smart routing personalizes recommendations progressively as users engage.
- Community review systems on existing platforms lack social features such as review upvotes, user profiles, and review history.
- Trending and box-office analytics are rarely surfaced in a unified interface alongside catalogue browsing.

1.3 Target Users

CineVault is designed for three primary user segments:

- Movie enthusiasts who actively track their watch history, rate films, and seek intelligent recommendations.
- Students and researchers who need structured, queryable movie data with ratings, genres, directors, and box office figures.
- Casual viewers who want a clean, ad-free interface to browse trending films and build a watchlist without signing up for multiple OTT subscriptions.

1.4 Key Differentiators

- Adaptive ML recommendation engine that intelligently switches between three strategies — content-based, collaborative, and cold-start — based on user engagement data.
- Unified platform covering discovery, watchlist status tracking (Want to Watch / Watching / Completed), reviews, and analytics.

- Social review layer with review likes, user profiles with aggregated stats, and paginated community review feeds.
- Real-time trending engine powered by view event logging and score computation on every movie page load.

2. Product Vision & Goals

2.1 Vision Statement

"To be the most complete, community-driven movie intelligence platform — where every user, regardless of their engagement level, receives a personalized, data-rich entertainment discovery experience."

2.2 Short-Term Goals (MVP)

1. Deliver a fully functional movie browsing platform with search, filter, and sort capabilities across genre, year, IMDb score, and release year.
2. Implement a secure, session-based authentication system with bcrypt password hashing, profile management, and password change functionality.
3. Enable users to manage a personal watchlist with three-state status tracking (Want to Watch, Watching, Completed).
4. Build a community review system with 1-10 rating scale, text comments, review likes, and paginated feeds.
5. Deploy a multi-strategy recommendation engine that adapts its algorithm based on user engagement history.
6. Surface real-time trending and box office analytics on the home page.

2.3 Long-Term Scalability Goals

- Integrate external data sources (TMDb API, OMDb API) to provide a continuously updated, larger movie catalogue.
- Evolve the recommendation engine using pre-trained ML models (collaborative filtering via matrix factorization, neural collaborative filtering).
- Introduce an editorial and curated content system where administrators publish movie reviews, director spotlights, and seasonal lists.
- Expand to a mobile application (React Native or Flutter) with push notifications for new releases and recommendation updates.
- Implement advanced analytics dashboards for administrators to monitor platform usage, popular genres, and user engagement patterns.
- Enable real-time features (WebSockets or Server-Sent Events) for live trending updates and notification feeds.

2.4 Success Metrics

User Engagement	Average session length > 5 minutes; watchlist additions per active user > 3
Review Quality	Average review length > 50 words; review likes per review > 1.5
Recommendation CTR	Click-through on recommended movies > 25% for logged-in users
System Performance	All API endpoints respond in < 300ms on local MySQL instance

3. Features & Functional Requirements

3.1 Authentication System

3.1.1 User Registration

- Accepts: username (3-50 characters, alphanumeric + underscore), email (RFC 5322 format), password (minimum 6 characters).
- Server-side validation rejects duplicate usernames and email addresses with distinct HTTP 409 Conflict responses.
- Passwords are hashed using bcrypt with a randomly generated salt before storage — raw passwords are never persisted.
- On successful registration, the user is automatically logged in and a server-side session is created.

3.1.2 Login & Session Management

- Email-based login with bcrypt password verification. Invalid credentials return a generic error to prevent user enumeration.
- Sessions are stored server-side using Flask's signed cookie mechanism. The session cookie is HttpOnly and SameSite=Lax for XSS and CSRF protection.
- GET /api/auth/me enables the frontend to check authentication state on page load without requiring re-login.

3.1.3 User Profile

- Users can update their bio (max 500 characters) and avatar URL. Public profiles display review count, watchlist count, and average rating given.
- Password change requires verification of the current password before the new hash is stored.

3.2 Movie Browsing & Discovery

- The movie catalogue displays all films with: title, release year, PG rating, duration, IMDb score, Metascore, genres, director, and poster image.

- Filtering: users can filter the catalogue by genre (using a subquery on the many-to-many junction table) and by release year.
- Sorting: the catalogue supports server-side sorting by release year, IMDb score, Metascore, and title — all validated against a whitelist to prevent SQL injection.
- Featured movie: a single hero movie is flagged in the database (is_featured = TRUE) and rendered prominently on the home page with trailer link and full cast.
- Movie detail page: a dedicated page (/movie/<id>) loads full movie data including cast, director, box office gross, trailer URL, and similar movie recommendations.
- Each movie detail page fetch automatically increments the trending score and logs a view event for analytics.

3.3 Search Functionality

- Full-text search across four fields: title, plot synopsis, director name, and cast list.
- Results are ranked with title matches surfaced first, followed by relevance by IMDb score.
- Minimum query length of 2 characters enforced server-side. Results capped at 20 for performance.
- Search is performed using SQL LIKE queries with parameterized inputs — no raw string interpolation.

3.4 Reviews & Ratings

- Users submit reviews with a numeric rating (1-10 scale) and an optional text comment (max 2,000 characters).
- One review per user per movie is enforced at the database level using a UNIQUE constraint (UNIQUE KEY one_review_per_user). Submitting again updates the existing review (ON DUPLICATE KEY UPDATE).
- Reviews are displayed in a paginated feed sorted by likes count (descending), then by recency. Pagination is controlled via ?page= and ?per_page= parameters.
- Each movie's review page shows the aggregate average rating and total review count alongside individual reviews.
- Review likes: users can toggle a like on any review. The system uses a review_likes junction table to prevent double-liking. The likes_count on the review row is maintained denormalized for fast display.
- Users can delete their own reviews. Review ownership is enforced server-side by matching session user_id.

3.5 Watchlist Management

- Users can add any movie to their personal watchlist with an initial status of want_to_watch.
- Three status states are supported: Want to Watch, Watching, and Completed — tracked via a MySQL ENUM column.
- Status can be updated at any time via PUT /api/watchlist/status/<movie_id>, enabling a Kanban-style progress tracker.
- Watchlist supports filtering by status (?status=completed) to display curated views.

- A quick-check endpoint (GET /api/watchlist/check/<movie_id>) allows the movie detail page to instantly show whether a film is already in the user's list without loading the full watchlist.
- Duplicate entries are handled via ON DUPLICATE KEY UPDATE — adding an existing movie updates its status rather than creating a duplicate row.

3.6 Recommendation System

The recommendation engine is the most technically sophisticated component of CineVault. It implements a smart routing architecture that selects between three distinct recommendation strategies based on the user's engagement profile. Detailed technical treatment appears in Section 9.

- Strategy 1 — Genre-Based (Content Filtering): Triggered when watchlist has 3+ entries.
- Strategy 2 — Collaborative Filtering: Triggered when review count has 3+ entries but watchlist is sparse.
- Strategy 3 — Top-Rated (Cold Start): Fallback for new or unauthenticated users.
- Supplementary — Similar Movies: Genre-overlap similarity for a specific source movie (More Like This).
- Supplementary — By Director: All films by the same director, ordered by IMDb score.

3.7 Trending & Analytics

- Trending scores are computed dynamically: each GET /api/movies/<id> call increments both view_count and score in the trending table using an upsert (INSERT ... ON DUPLICATE KEY UPDATE).
- Top 10 trending movies are exposed via GET /api/movies/trending, joining the trending table with movie details and genres.
- Box office rankings are stored in the box_office table with week_of date, rank, and gross — accessible via GET /api/movies/boxoffice.
- Editorial content (director spotlights, upcoming releases, award season coverage) is managed in the editorials table and surfaced via GET /api/movies/editorials.

4. User Personas

4.1 Persona A — The Enthusiast

Name	Arjun Mehta, 23, Computer Science Student
Goals	Track every movie he watches; discover films similar to his taste; share opinions through reviews
Behaviour	Logs in daily; has a 30+ movie watchlist; rates every film after watching; values recommendation quality

Pain Point	Existing platforms don't let him track watch status AND get personalized recommendations AND read community reviews in one place
CineVault Value	Full watchlist with status tracking + adaptive recommendations + review social features

4.2 Persona B — The Casual Viewer

Name	Priya Sharma, 35, Marketing Professional
Goals	Quickly find a good movie to watch on weekends; see what's trending; read a few reviews before committing
Behaviour	Visits 1-2 times per week; rarely writes reviews; uses search and trending sections; low friction preferred
Pain Point	IMDb is data-heavy and complex; she wants a clean interface showing trending and high-rated films
CineVault Value	Clean home page with trending, featured film, and top-rated recommendations — no login needed

4.3 Persona C — The Critic

Name	Rohan Das, 27, Film Studies Graduate
Goals	Write detailed film reviews; build a public profile; discover critically acclaimed films by IMDb and Metascore
Behaviour	Writes long review comments; sorts by Metascore; explores editorials; checks other users' review histories
Pain Point	Wants a platform that surfaces both community ratings and critic scores side-by-side with a social review layer
CineVault Value	IMDb score + Metascore dual scoring, review likes system, public user profiles with review history

5. User Flow Diagrams

5.1 New User Onboarding Flow

7. User lands on home page → featured movie and trending section visible without login (cold-start recommendations shown).
8. User clicks Register → Registration form modal appears → submits username, email, password.
9. Backend validates all fields (length, format, uniqueness) → creates user record → stores bcrypt hash → creates session.
10. User is redirected back to home page — now logged in. Recommendation strategy upgrades from top-rated to watchlist/review-based as user engages.

5.2 Movie Discovery & Watchlist Flow

11. Logged-in user browses catalogue → applies genre filter (e.g., Sci-Fi) → sorts by IMDb score DESC.
12. User clicks on a movie card → navigates to /movie/<id> → Flask serves movie.html → JS reads ID from URL → calls GET /api/movies/<id>.
13. Movie detail loads: cast, plot, genres, scores, trailer link, and similar recommendations appear.
14. User clicks Add to Watchlist → POST /api/watchlist/add with status want_to_watch.
15. Later, user opens watchlist → updates status to watching via PUT /api/watchlist/status/<movie_id>.
16. After watching, user updates status to completed → submits review with rating 8/10 + text comment.

5.3 Recommendation Discovery Flow

17. New user (< 3 watchlist entries, < 3 reviews): GET /api/recommendations/ → smart router returns top-rated strategy.
18. User adds 3 movies to watchlist: GET /api/recommendations/ → router detects watchlist_count >= 3 → switches to genre-based strategy.
19. User writes 3+ reviews: collaborative filtering becomes eligible (router checks review_count >= 3, watchlist_count < 3 triggers it).
20. On any movie page, 'More Like This' section calls GET /api/recommendations/similar/<id> → returns genre-overlap ranked films.
21. 'More by Director' section calls GET /api/recommendations/by-director/<id> → returns director's other films ordered by score.

5.4 Social Review Flow

22. User navigates to a movie's review section → paginated reviews loaded from GET /api/reviews/<movie_id>?page=1&per_page=10.
23. User sees another reviewer's detailed comment → clicks thumbs up → POST /api/reviews/like/<review_id>.
24. Backend checks review_likes table for duplicates → inserts like or removes it (toggle) → updates likes_count denormalized counter.
25. User clicks on reviewer's username → navigates to their public profile → GET /api/auth/profile/<user_id> returns bio, stats, and review history.

6. System Architecture

6.1 Architecture Overview

CineVault follows a classic three-tier web application architecture: a static frontend tier, a RESTful API middleware tier (Flask), and a relational data persistence tier (MySQL). The ML recommendation logic is embedded within the Flask middleware tier as a set of SQL-driven algorithms, not as a separate microservice.

6.2 Frontend Tier

- Technology: HTML5, CSS3 (with Tailwind-style utility classes), Vanilla JavaScript (ES6+).
- Structure: Two primary HTML files served by Flask as static assets — index.html (home/catalogue) and movie.html (movie detail page). All routing is handled server-side via Flask's @app.route decorators.
- API Communication: All data fetching uses the browser's native fetch() API with async/await syntax. The frontend is decoupled from the backend — it only receives JSON and renders it dynamically into the DOM.
- Session State: The frontend calls GET /api/auth/me on load to determine authentication state. UI elements (Add to Watchlist, Review Form, Recommendations) are conditionally rendered based on login status.
- No frontend framework dependency: the choice of Vanilla JS eliminates build toolchain complexity, making the project easy to run locally without Node.js, webpack, or npm.

6.3 Backend Tier — Flask Application

- Application Factory Pattern: create_app() in app.py initializes all extensions and registers blueprints. This pattern enables environment-specific configuration and simplifies unit testing.
- Blueprint Architecture: the application is organized into five domain-specific blueprints — auth, movies, reviews, watchlist, and recommendations — each prefixed with /api/<domain>. This enforces separation of concerns and makes the codebase navigable.
- Extensions: Flask-MySQLdb provides MySQL connectivity; Flask-CORS handles cross-origin headers (configured to localhost:5000). Extensions are initialized in extensions.py to avoid circular imports — a standard Flask best practice.
- Session Management: Flask's server-side session stores user_id and username in a signed, encrypted cookie. The @login_required decorator (implemented as a Python functools.wraps closure) protects all authenticated endpoints.
- Input Validation: all route handlers validate input type (int/str), length, format (regex for email), and range (1-10 for ratings) before any database operation. Parameterized queries are used throughout — no string interpolation in SQL.

6.4 Database Tier — MySQL

- Database: MySQL 8.x with the InnoDB storage engine for full ACID compliance and foreign key enforcement.
- Schema: 11 tables organized around core entities (users, movies, genres) and their relationships (movie_genres junction, watchlist, reviews, review_likes, trending, movie_views, box_office, editorials).
- Connection: Flask-MySQLdb creates a connection per request (connection pooling via the underlying MySQLdb C extension). DictCursor is configured globally so all query results are Python dictionaries.
- Query Strategy: complex, multi-table data (e.g., movies with their genre list) uses LEFT JOINs with GROUP_CONCAT to aggregate genre names in a single query, avoiding the N+1 query problem.

6.5 Request-Response Cycle

A full request lifecycle for GET /api/movies/?genre=Sci-Fi&sort=imdb_score&order=DESC:

26. Browser sends HTTP GET request with query parameters.
27. Flask's URL dispatcher matches the route to movies_bp.get_all_movies().
28. Route handler validates query params: checks genre is a string, year is numeric, sort is whitelisted, order is ASC/DESC.
29. Parameterized SQL query executes against MySQL — joins movies, movie_genres, and genres tables with optional WHERE clause.
30. MySQL returns dict rows via DictCursor → Flask serializes to JSON using jsonify().
31. HTTP 200 response with JSON body → JavaScript fetch() receives it → DOM is updated.

7. Database Design

CineVault uses a normalized relational schema (approximately 3NF) with 11 tables. Foreign key relationships enforce referential integrity, and CASCADE rules ensure orphaned records are automatically cleaned up.

Table: users

Column	Data Type	Constraints / Notes
`id`	INT	PRIMARY KEY, AUTO_INCREMENT
`username`	VARCHAR(50)	NOT NULL, UNIQUE — alphanumeric + underscore only
`email`	VARCHAR(100)	NOT NULL, UNIQUE — normalized to lowercase on insert
`password_hash`	VARCHAR(255)	NOT NULL — bcrypt hash, never raw password
`bio`	TEXT	Optional — max 500 characters enforced in API layer
`avatar_url`	VARCHAR(500)	Optional — URL to user's profile image
`is_admin`	BOOLEAN	DEFAULT FALSE — reserved for admin dashboard (future)

created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

Table: movies

Column	Data Type	Constraints / Notes
id	INT	PRIMARY KEY, AUTO_INCREMENT
title	VARCHAR(200)	NOT NULL
release_year	INT	NOT NULL
rating_pg	VARCHAR(10)	G / PG / PG-13 / R / NR
duration_min	INT	Runtime in minutes
imdb_score	DECIMAL(3,1)	0.0 to 10.0 — e.g., 8.4
metascore	INT	0-100 — critical consensus score
plot	TEXT	Synopsis / description
poster_url	VARCHAR(500)	External URL to poster image
trailer_url	VARCHAR(500)	YouTube or external trailer link
director	VARCHAR(200)	Used by by-director recommendations
cast_list	TEXT	Comma-separated cast — searched by search endpoint
language	VARCHAR(100)	DEFAULT English
country	VARCHAR(100)	DEFAULT USA
box_office_gross	VARCHAR(50)	Formatted string e.g., $2.798B
is_featured	BOOLEAN	Marks the hero featured movie (max 1)

Table: genres & movie_genres

Column	Data Type	Constraints / Notes
genres.id	INT	PRIMARY KEY — e.g., 1=Action, 2=Adventure
genres.name	VARCHAR(50)	UNIQUE — seeded with 12 genres on schema load
movie_genres.movie_id	INT	FK → movies.id ON DELETE CASCADE
movie_genres.genre_id	INT	FK → genres.id ON DELETE CASCADE
(movie_id, genre_id)	COMPOSITE PK	Prevents duplicate genre assignments

Table: watchlist

Column	Data Type	Constraints / Notes
id	INT	PRIMARY KEY, AUTO_INCREMENT
user_id	INT	FK → users.id ON DELETE CASCADE
movie_id	INT	FK → movies.id ON DELETE CASCADE
status	ENUM	want_to_watch | watching | completed — DEFAULT want_to_watch
added_at	DATETIME	DEFAULT CURRENT_TIMESTAMP
UNIQUE(user_id, movie_id)	CONSTRAINT	Prevents duplicate watchlist entries — ON DUPLICATE KEY UPDATE handles re-adds

Table: reviews

Column	Data Type	Constraints / Notes
`id`	INT	PRIMARY KEY, AUTO_INCREMENT
`user_id`	INT	FK → users.id ON DELETE CASCADE
`movie_id`	INT	FK → movies.id ON DELETE CASCADE
`rating`	TINYINT	CHECK (rating BETWEEN 1 AND 10)
`comment`	TEXT	Optional — max 2000 chars enforced in API
`likes_count`	INT	DEFAULT 0 — denormalized for fast ordering
`UNIQUE(user_id, movie_id)`	CONSTRAINT	One review per user per movie

Table: review_likes

Column	Data Type	Constraints / Notes
`user_id`	INT	FK → users.id ON DELETE CASCADE
`review_id`	INT	FK → reviews.id ON DELETE CASCADE
`liked_at`	DATETIME	DEFAULT CURRENT_TIMESTAMP
`(user_id, review_id)`	COMPOSITE PK	Prevents double-liking — idempotent toggle

Table: trending & movie_views

Column	Data Type	Constraints / Notes
`trending.movie_id`	INT	UNIQUE FK → movies.id — one row per movie
`trending.score`	INT	Incremented on every movie detail page fetch
`trending.view_count`	INT	Raw page view counter
`trending.updated_at`	DATETIME	ON UPDATE CURRENT_TIMESTAMP — auto-refreshes
`movie_views.movie_id`	INT	FK — event log per view
`movie_views.user_id`	INT	NULL for anonymous viewers
`movie_views.viewed_at`	DATETIME	Timestamp for analytics

8. API Design

All API endpoints follow RESTful conventions. Responses are JSON with a consistent structure: { "success": true/false, "data": ..., "error": "..." }. All authenticated routes return HTTP 401 if no valid session is present.

8.1 Authentication API — /api/auth

Method	Endpoint	Description	Auth Required
POST	/api/auth/register	Create new user. Validates email, username, password.	No
POST	/api/auth/login	Authenticate user. Returns session cookie on success.	No
POST	/api/auth/logout	Clear server-side session.	Yes
GET	/api/auth/me	Return current logged-in user ID and username.	No
GET	/api/auth/profile/:user_id	Public profile: bio, avatar, review count, avg rating, watchlist count.	No
PUT	/api/auth/profile	Update own bio and avatar URL.	Yes
POST	/api/auth/change-password	Verify current password then store new bcrypt hash.	Yes

8.2 Movies API — /api/movies

Method	Endpoint	Description	Auth Required
GET	/api/movies/	All movies. Params: ?genre= ?year= ?sort= ?order=	No
GET	/api/movies/search?q=	Full-text search: title, plot, director, cast. Min 2 chars.	No
GET	/api/movies/featured	Single hero movie (is_featured=TRUE).	No
GET	/api/movies/trending	Top 10 by trending score (view_count + score).	No
GET	/api/movies/boxoffice	Box office weekly rankings.	No
GET	/api/movies/editorials	Latest 3 editorial articles.	No
GET	/api/movies/genres	All genre names (for filter UI dropdowns).	No
GET	/api/movies/:id	Full movie detail. Also bumps trending score + logs view.	No

8.3 Watchlist API — /api/watchlist

Method	Endpoint	Description	Auth Required
POST	/api/watchlist/add	Add movie with status. ON DUPLICATE KEY UPDATE status.	Yes
GET	/api/watchlist/	All watchlist entries. Supports ?status= filter.	Yes
PUT	/api/watchlist/status/:movie_id	Update watch status (want_to_watch/watching/completed).	Yes
DELETE	/api/watchlist/remove/:movie_id	Remove movie from watchlist.	Yes

GET	/api/watchlist/check/:movie_id	Is this movie in my watchlist? Returns status.	Yes

8.4 Reviews API — /api/reviews

Method	Endpoint	Description	Auth Required
POST	/api/reviews/add	Submit review (rating 1-10, comment max 2000 chars).	Yes
GET	/api/reviews/:movie_id	Paginated reviews. Params: ?page= ?per_page=. Includes avg rating.	No
DELETE	/api/reviews/:review_id	Delete own review (ownership enforced by session).	Yes
POST	/api/reviews/like/:review_id	Toggle like/unlike a review. Idempotent.	Yes
GET	/api/reviews/user/:user_id	All reviews by a specific user (for public profiles).	No
GET	/api/reviews/my/:movie_id	My own review for a movie (pre-fill review form).	Yes

8.5 Recommendations API — /api/recommendations

Method	Endpoint	Description	Auth Required
GET	/api/recommendations/	Smart router: auto-selects best strategy for current user.	No
GET	/api/recommendations/genre-based	Content-based: genre overlap with user's watchlist.	Yes
GET	/api/recommendations/collaborative	User-based CF: movies liked by similar reviewers.	Yes
GET	/api/recommendations/top-rated	Cold-start fallback: highest community-rated films.	No
GET	/api/recommendations/similar/:id	More Like This: genre-overlap similarity to source movie.	No
GET	/api/recommendations/by-director/:id	All films by same director, ordered by IMDb score.	No

8.6 API Response Structure

All API responses conform to the following consistent structure:

Success Response	{ "success": true, "count": N, "data_key": [...] }

Error Response	{ "success": false, "error": "Human-readable message" }

HTTP Status Codes	200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 409 Conflict, 500 Server Error

9. Machine Learning Integration

9.1 Overview

CineVault's recommendation engine implements a layered, SQL-driven machine learning pipeline that delivers personalized recommendations without requiring a separate ML microservice. The recommendation logic is embedded directly within the Flask blueprint (blueprints/recommendations.py), making it testable, maintainable, and deployable without additional infrastructure.

The system employs three complementary strategies: content-based filtering, user-based collaborative filtering, and a cold-start handler — all orchestrated by an intelligent routing layer that selects the optimal strategy based on each user's engagement profile.

9.2 Strategy 1 — Content-Based Filtering (Genre Overlap)

Trigger Condition

Activated when the user has 3 or more movies in their watchlist.

Algorithm

The algorithm extracts the set of genre IDs from all movies in the user's watchlist. It then identifies all movies that share at least one genre with this set, excludes movies already in the watchlist, and ranks results by the count of shared genres (match_score) followed by IMDb score as a tiebreaker.

SQL Implementation

- A correlated subquery extracts the unique genre IDs from the user's watchlist via a JOIN on movie_genres.
- The outer query matches candidate movies on these genre IDs using a JOIN (not a subquery in the WHERE clause), enabling the COUNT(DISTINCT genre_id) aggregate to produce the match_score.
- An exclusion subquery (NOT IN watchlist) ensures already-seen content is not recommended.

Strengths & Limitations

- Strength: works with zero cross-user data; pure preference-based. Does not suffer from data sparsity.
- Limitation: does not account for the user's rating quality — a movie added to watchlist but rated 2/10 still influences genre weights.

9.3 Strategy 2 — User-Based Collaborative Filtering

Trigger Condition

Activated when the user has 3 or more reviews submitted but fewer than 3 watchlist entries.

Algorithm

This strategy identifies similar users (users who have rated the same movies with ratings within 2 points of the current user's ratings) and then recommends movies that those similar users have rated 7 or higher which the current user has not yet reviewed.

SQL Implementation

- Self-join on the reviews table (r1 = current user, r2 = other users) where r1.movie_id = r2.movie_id AND ABS(r1.rating - r2.rating) <= 2 identifies rating-similar users.
- A second join (r3) retrieves the highly-rated movies (rating >= 7) from those similar users.
- Results are aggregated by community_score (average rating from similar users) and similar_users_count, giving confidence weighting.
- HAVING similar_users_count >= 1 ensures at least one corroborating similar user.

Strengths & Limitations

- Strength: discovers serendipitous content outside the user's own genre preferences — the hallmark of collaborative filtering.
- Limitation: subject to cold-start for new users and sparse data; requires the platform to accumulate review data across users.

9.4 Strategy 3 — Cold-Start (Top-Rated Fallback)

Trigger Condition

Activated for all unauthenticated users and for new users with fewer than 3 watchlist entries and fewer than 3 reviews.

Algorithm

Returns the 6 movies with the highest combined score — computed as COALESCE(AVG(community_rating), imdb_score) — ensuring that movies with no community ratings fall back to their IMDb score rather than being excluded.

This is a popularity-based recommendation strategy, a standard industry approach for handling the cold-start problem.

9.5 Smart Routing Logic

The GET /api/recommendations/ endpoint implements a decision tree that runs two COUNT queries before selecting a strategy:

32. Query watchlist count for current user.
33. Query review count for current user.
34. If watchlist_count >= 3: use genre-based (content filtering).
35. Else if review_count >= 3: use collaborative filtering.
36. Else if watchlist_count > 0: use genre-based (partial data).
37. Else (no data or anonymous): use top-rated (cold start).

This progressive enhancement model ensures every user receives the most relevant recommendations possible given their current engagement level — a technique inspired by Netflix's recommendation maturity model.

9.6 Supplementary Recommendation Modes

Similar Movies (More Like This)

For a given source movie, the algorithm counts shared genre IDs with every other movie in the catalogue and returns the top 4 by shared_genres count, with IMDb score as tiebreaker. This powers the 'More Like This' section on every movie detail page.

By Director

Retrieves all other movies by the same director, ordered by IMDb score. This surfaces a director's filmography, supporting cinephiles who follow auteur directors.

9.7 ML Data Flow Diagram (Textual)

The end-to-end data flow for a personalized recommendation request:

38. User's browser calls GET /api/recommendations/.
39. Flask smart router queries MySQL for user's watchlist_count and review_count.
40. Router selects strategy (genre-based / collaborative / top-rated).
41. Strategy function executes a multi-table JOIN query against MySQL — leveraging movie_genres, watchlist, reviews, and genres tables.
42. MySQL returns up to 6 ranked movie records as dict rows.
43. Flask serializes results to JSON: { strategy, reason, count, movies: [...] }.
44. Browser JavaScript renders recommendation cards, displaying the strategy reason (e.g., 'Based on genres in your watchlist') for transparency.

9.8 Future ML Enhancements

- Matrix Factorization (SVD/ALS): decompose the user-movie rating matrix to produce latent factor embeddings. Libraries such as Surprise or LightFM can be integrated as a separate prediction service called by the recommendations blueprint.
- Neural Collaborative Filtering (NCF): use embedding layers to learn non-linear user-item interactions, deployed via Flask API calling a PyTorch/TensorFlow model.
- Pre-Trained Model Integration (.whl): the project architecture supports loading pre-trained recommendation models packaged as Python wheels. The blueprint can import these models and use them to score candidate movies, replacing or supplementing the SQL-based algorithms.
- Implicit Feedback Signals: incorporate view duration (from movie_views timestamps), watchlist additions, and review likes as implicit preference signals for richer collaborative filtering.

10. Non-Functional Requirements

10.1 Performance

- All API endpoints must respond within 300ms for catalogue queries on a local MySQL instance with up to 10,000 movie records.
- The recommendations endpoint may take up to 500ms due to multi-table self-join complexity; this is acceptable for a non-real-time feature.
- Movie detail pages execute 3 queries (movie data, trending upsert, view log insert) — these are kept sequential to ensure view count accuracy.
- GROUP_CONCAT for genre aggregation is performed at the SQL layer to avoid N+1 query patterns in the Python application layer.

10.2 Scalability

- The Application Factory pattern (create_app()) enables horizontal scaling — multiple Flask worker processes can be spawned behind a load balancer (Gunicorn + Nginx in production).
- The stateless JSON API design means any request can be handled by any worker — session state is in the client-side cookie, not in-process memory.
- MySQL connection pooling is handled by Flask-MySQLdb's underlying MySQLdb C extension.
- The trending system uses UPSERT semantics (INSERT ... ON DUPLICATE KEY UPDATE) to handle concurrent view increments without race conditions.

10.3 Security

- Authentication: bcrypt with per-password random salts. No raw passwords stored. No predictable hashing (MD5/SHA1 not used).
- Session: HttpOnly cookies prevent JavaScript session theft. SameSite=Lax prevents CSRF in cross-origin form submissions.
- SQL Injection: all database queries use parameterized inputs (%s placeholders with MySQLdb). No f-string or .format() interpolation into SQL.
- Input Validation: sort column whitelist prevents ORDER BY injection. All integer inputs (movie_id, rating) are explicitly cast and validated.
- CORS: restricted to localhost:5000 in development. In production, this would be locked to the application's domain.
- Secret Key: Flask's SECRET_KEY signs session cookies. Must be changed to a cryptographically random value in production.

10.4 Usability

- The frontend requires no JavaScript framework installation — runs in any modern browser by simply opening the served HTML page.
- All interactive actions (Add to Watchlist, Submit Review, Like) provide immediate UI feedback without full page reloads.
- Error messages are user-readable (e.g., 'Username must be 3-50 characters') rather than raw exception messages.

- The movie detail page extracts the movie ID from the URL path, so each film has a shareable, bookmarkable URL (/movie/7).

10.5 Maintainability

- Blueprint architecture ensures each feature domain (auth, movies, reviews, watchlist, recommendations) is encapsulated in its own module with clear API boundaries.
- All configuration (DB credentials, secret key, session settings) is centralized in config.py — no hardcoded values in route handlers.
- The schema.sql file includes both the DDL and SEED DATA, making the database reproducible from a single file.
- The README.md documents setup in 5 steps, enabling a fresh developer to run the project in under 10 minutes.

11. Tech Stack Justification

11.1 Python Flask

- Lightweight and minimal: Flask provides routing, request handling, and extension integration without imposing an MVC structure — ideal for academic projects where the team controls the architecture.
- Blueprint support enables modular, scalable application organization without migrating to a heavier framework like Django.
- Python's native integration with ML libraries (scikit-learn, TensorFlow, Surprise) makes Flask the natural choice for projects that will incorporate pre-trained models.
- Alternative considered: Django — rejected due to ORM overhead and template-centric design that conflicts with the JSON API architecture.

11.2 MySQL

- Relational data model is ideal for CineVault's structured, relationship-heavy domain: movies have many genres, users have many reviews, reviews have many likes.
- InnoDB's foreign key constraints and CASCADE rules enforce data integrity at the database layer — not just the application layer.
- GROUP_CONCAT and window functions enable efficient aggregation (genre lists, trending scores, collaborative filtering) without multiple round-trips.
- Alternative considered: PostgreSQL — equivalent capability; MySQL was chosen for its ubiquity in academic environments and WAMP/XAMPP local setups.

11.3 Vanilla JavaScript

- Zero build toolchain: no npm, webpack, or transpilation needed. Students can run the project with just Python installed.

- fetch() API with async/await provides modern, promise-based HTTP communication without any library dependency.
- DOM manipulation is explicit and transparent — students can trace every UI update to the exact JavaScript function, reinforcing learning.
- Alternative considered: React — rejected for increased complexity (Node.js dependency, JSX, state management) that would shift focus away from the backend and database design, which are the core learning objectives.

11.4 bcrypt

- Industry-standard password hashing with adaptive cost factor. Unlike MD5 or SHA, bcrypt is designed to be slow — making brute-force attacks computationally expensive.
- Per-password random salts prevent rainbow table attacks.

11.5 Flask-CORS

- Enables the frontend (served by Flask) to communicate with the API. In this project, both are on the same origin, but CORS is configured explicitly to demonstrate production-readiness awareness.

12. Current Limitations

12.1 Dataset Limitations

- The seed dataset contains only 10 movies across 10 entries in schema.sql. In production, the platform requires integration with a movie metadata API (TMDb, OMDb) or a larger pre-loaded dataset (e.g., MovieLens 25M).
- Genre taxonomy is limited to 12 predefined genres. Niche genres (Anime, Documentary, Short Film) are absent.
- Metascore and box office data are manually entered strings — not dynamically fetched from an authoritative source.

12.2 Recommendation System Limitations

- The collaborative filtering algorithm requires a minimum critical mass of reviews across users. With a small dataset or few active users, the self-join returns empty results and falls back to top-rated.
- Content-based filtering uses only genre as the feature vector. A richer model would incorporate director, cast, plot keywords (TF-IDF), language, and runtime.
- No matrix factorization or embedding-based approaches are implemented in v2 — these require pre-trained model files and inference infrastructure.
- Recommendation results are not cached — each request re-executes the multi-table JOIN queries.

12.3 System Limitations

- No email verification for new user registration — the email field is stored but not validated via a verification link.
- Session-based authentication does not support JWT tokens — unsuitable for mobile app or multi-tab scenarios where cookie sharing may be inconsistent.
- The trending score is a simple incrementing counter — it does not decay over time. A production system would implement a time-decay function (e.g., Reddit's decay algorithm).
- No rate limiting is implemented on the API endpoints — a DoS attack could overwhelm the MySQL connection pool.
- Image/poster data relies on external URLs (Google CDN, TMDb). If external URLs break, poster images will fail to load.

13. Future Enhancements

13.1 AI & ML Enhancements

- Matrix Factorization: implement SVD or ALS-based collaborative filtering using the Surprise library. Store user and movie latent factor embeddings in a recommendations_cache table for fast retrieval.
- NLP-Based Content Filtering: apply TF-IDF or Word2Vec on movie plot summaries to compute semantic similarity between films — enabling plot-based recommendations beyond genre overlap.
- Neural Collaborative Filtering: train an NCF model on the user-movie rating matrix using PyTorch. Package as a .whl and load in the Flask recommendations blueprint.
- Real-Time Recommendations: implement a streaming pipeline (Kafka + Faust) to update recommendation scores in near-real-time based on view events.

13.2 Feature Enhancements

- User Lists: allow users to create named curated lists (e.g., 'Best Nolan Films') and share them publicly.
- Advanced Search: add filters for language, country, director, cast, runtime, and Metascore range.
- Notification System: email or in-app notifications for new movies in a user's preferred genres, or when a followed user posts a review.
- Social Following: allow users to follow other users and see a feed of their reviews and watchlist additions.
- Administrator Dashboard: content moderation panel for managing movies, editorials, and featured content. Powered by the is_admin flag in the users table.

13.3 Platform Enhancements

- Mobile Application: React Native or Flutter app with deep linking to /movie/<id> URLs, push notifications for trending alerts, and offline watchlist viewing.
- External API Integration: TMDb API integration to auto-populate movie metadata, posters, and trailers on demand — eliminating the manual data entry bottleneck.
- Trending Decay Algorithm: implement time-weighted trending scores (score = raw_views / (hours_since_view + 2)^gravity) similar to Hacker News ranking.
- Review Analytics: word cloud of review keywords per movie; sentiment analysis on review text using VADER or BERT.

14. Deployment Strategy

14.1 Local Development (Current)

45. Install Python dependencies: pip install flask flask-mysqldb flask-cors bcrypt
46. Set up MySQL: mysql -u root -p < schema.sql
47. Configure database password in config.py
48. Start Flask server: python app.py
49. Access at http://localhost:5000/

14.2 Production Deployment (Recommended Path)

Phase 1 — Containerization

- Dockerize the Flask application with a Dockerfile. Use docker-compose to orchestrate Flask + MySQL containers with persistent volume for DB data.
- Set SECRET_KEY, MYSQL_PASSWORD, and MYSQL_HOST as environment variables — never hardcoded.

Phase 2 — Cloud Deployment

- Deploy Flask behind Gunicorn (WSGI server) + Nginx (reverse proxy for static file serving and SSL termination).
- Host on AWS EC2 (t3.micro free tier) or Google Cloud Run (serverless containers) for scalable, managed deployment.
- Use AWS RDS (MySQL) or PlanetScale (MySQL-compatible serverless DB) for the database tier.
- Configure CORS to the production domain and update SESSION_COOKIE_SECURE = True and SESSION_COOKIE_SAMESITE = 'Strict'.

Phase 3 — Monitoring & Reliability

- Add structured logging (Python logging module → CloudWatch or Stackdriver).
- Implement health check endpoint (GET /api/health) returning DB ping status.

- Set up uptime monitoring (UptimeRobot or Pingdom) and error alerting (Sentry for Flask exception tracking).

14.3 Environment Configuration Matrix

Setting	Development	Production	Notes
DEBUG	True	False	Never expose in prod
SECRET_KEY	cinevault_secret	Random 32-byte hex	Use secrets.token_hex(32)
MYSQL_HOST	localhost	RDS endpoint URL	
CORS Origins	localhost:5000	https://cinevault.app	Lock to production domain
COOKIE_SECURE	False	True	Requires HTTPS

15. Conclusion

15.1 Project Impact

CineVault successfully demonstrates that a production-grade, feature-rich web application can be built on a minimal, well-structured technology stack. The project delivers real value across five distinct feature domains — authentication, movie discovery, reviews, watchlist management, and intelligent recommendations — all integrated into a cohesive user experience.

The multi-strategy recommendation engine is the most technically sophisticated component of the project. By implementing content-based filtering, user-based collaborative filtering, and a cold-start handler — all within SQL queries orchestrated by a smart routing layer — CineVault demonstrates an understanding of recommendation system design that goes beyond the typical final-year project scope.

15.2 Technical Learning Outcomes

- Relational Database Design: 11-table normalized schema with junction tables, ENUM columns, composite primary keys, CASCADE rules, and analytical tables (trending, movie_views).
- RESTful API Architecture: Blueprint-based Flask application with consistent HTTP status codes, parameterized SQL, and session-based authentication.
- Recommendation Systems: practical implementation of content-based filtering, user-based collaborative filtering, and cold-start handling — with a smart routing decision layer.
- Security Engineering: bcrypt password hashing, HttpOnly/SameSite cookies, SQL injection prevention via parameterized queries, and input validation with whitelist patterns.
- Software Architecture: Application Factory pattern, separation of concerns via blueprints, extension isolation to prevent circular imports, and configuration centralization.

15.3 Academic Significance

CineVault integrates concepts from multiple Computer Science disciplines taught across a typical undergraduate programme: Database Management Systems (schema design, query optimization, ACID transactions), Web Technologies (REST APIs, HTTP, session management), Machine Learning (filtering algorithms, cold-start problem, similarity metrics), and Software Engineering (design patterns, modular architecture, security best practices).

This convergence of disciplines in a single, working application demonstrates the ability to synthesize academic knowledge into a practical, deployable system — the fundamental goal of a final-year capstone project.

15.4 Version History

Version	Date	Key Changes
v1.0	Initial Release	Core auth, movie CRUD, basic reviews, simple watchlist, top-rated recommendations only.
v2.0	Final Submission	Added: user profiles, bio/avatar, change-password, review likes, watchlist status, trending engine, movie_views log, genre-based and collaborative filtering, by-director recommendations, search, box office, editorials, paginated reviews.

— End of Document —